Default Stages

30 March 2024 08:14

.pre Stage:
- The .pre stage is a special stage in GitLab CI/CD pipelines that runs before any other stages.
- It can be used for tasks such as setting up environment variables, initializing dependencies, or performing pre-build checks.
- Tasks in this stage are executed once before the main pipeline stages start.

Build Stage:
- The build stage is where the source code is compiled, dependencies are resolved, and artifacts are generated.
- It typically includes tasks like compiling code, running build scripts, and packaging the application for deployment.
- The success of this stage indicates that the code can be built successfully and is ready for testing.

Test Stage:
- The test stage involves running various types of tests to verify the functionality and quality of the code.
- It can include unit tests, integration tests, end-to-end tests, and other forms of automated testing.
- The goal of this stage is to identify any bugs or issues in the codebase before it is deployed to production.

Deploy Stage:
- The deploy stage is where the built artifacts are deployed to the target environment, such as a testing, staging, or production server.
- It includes tasks like transferring files, configuring servers, and setting up the application environment.
- The success of this stage indicates that the application has been successfully deployed to the target environment.

.post Stage:
- The .post stage is another special stage in GitLab CI/CD pipelines that runs after all other stages have completed.
- It can be used for tasks such as cleanup operations, sending notifications, or performing post-deployment checks.
- Tasks in this stage are executed once after all the main pipeline stages have finished.

These stages provide a structured approach to building, testing, deploying, and managing applications in GitLab CI/CD pipelines. They can be customized and extended to fit the specific requirements of each project.

Order of .pre and .post cannot change, while for build, test and deploy order can be changed.

Script

30 March 2024 08:44

Script Section in GitLab CI/CD:

- It defines the docker commands or shell scripts to be executed within each job of the pipeline.
- Each job in the pipeline can have its own `script` section, allowing for customization and flexibility.
- The `script` section is where you specify the actions to be taken to accomplish the goals of the job, such as building, testing, deploying, etc.

```
stages:
  - build
  - test
  - deploy

build_job:
  stage: build
  script:
    - echo "Building the project..."
    - mvn clean install

test_job:
  stage: test
  script:
    - echo "Running tests..."
    - mvn test

deploy_job:
  stage: deploy
  script:
    - echo "Deploying the application..."
    - mvn deploy
```

Before and After Script

30 March 2024 08:47

- These sections allow for configuring actions to be performed before and after executing the main script of a job.

- The `before_script` section is executed before the main script of each job in the pipeline. It is commonly used for setting up the environment, installing dependencies, or initializing variables.

- Conversely, the `after_script` section is executed after the main script of each job completes. It is typically used for cleanup tasks, gathering logs, or sending notifications.

- Both `before_script` and `after_script` sections can contain multiple commands or scripts, which are executed sequentially.

Example:

```yaml
stages:
  - build
  - test

build_job:
  stage: build
  before_script:
    - echo "Setting up environment..."
    - apt-get update && apt-get install -y some-package
  script:
    - echo "Building the project..."
    - mvn clean install
  after_script:
    - echo "Cleaning up..."
    - rm -rf some-directory

test_job:
  stage: test
  before_script:
    - echo "Preparing for testing..."
  script:
    - echo "Running tests..."
    - mvn test
  after_script:
    - echo "Test cleanup..."
```

Variables

30 March 2024 08:51

1. Global Variables:
- Defined at the project level.
- Accessible to all pipelines within the project.
- Configured in GitLab project settings under "CI/CD > Variables".
- Used for storing non-sensitive information like environment configurations, URLs, etc.
- Values are visible in the job logs and can be accessed by anyone with access to the project settings.
- Examples include environment-specific configurations, API URLs, version numbers, etc.

2. Pipeline Variables:
  - Defined within a specific pipeline.
  - Accessible to jobs within that pipeline only.
  - Set using the `variables` keyword in the `.gitlab-ci.yml` file.
  - Useful for customizing individual pipeline behavior.

3. Job Variables:
  - Scoped to a specific job within a pipeline.
  - Defined using the `variables` keyword within a job configuration in `.gitlab-ci.yml`.
  - Useful for defining environment-specific configurations or overriding global/pipeline variables for specific jobs.

4. Environment Variables:
  - Predefined variables provided by GitLab.
  - Automatically available within the context of a pipeline or job.
  - Provide information about the pipeline, job, or runner environment.
  - Custom environment variables can also be defined using the `environment` keyword in `.gitlab-ci.yml`.

5. Secret Variables:
- Global variables specifically designed for storing sensitive information.
- Configured in GitLab project settings under "CI/CD > Variables", marked as "Protected" and "Masked".
- Encrypted and exposed only to the CI/CD runner during job execution.
- Used for storing credentials, tokens, or other confidential information securely.
- Values are hidden in the job logs and are not accessible by anyone viewing the logs.
- Examples include API tokens, database passwords, SSH keys, etc.

These variables provide flexibility and control over GitLab CI/CD pipeline configuration, enabling secure management of sensitive information and customization of pipeline execution.

```
stages:
  - build
  - test
  - deploy

variables:
  # Global variables
  GLOBAL_VAR: "This is a global variable"

  # Environment-specific variables
  DEV_VARIABLE: "This is a development environment variable"
  STAGING_VARIABLE: "This is a staging environment variable"
  PROD_VARIABLE: "This is a production environment variable"

before_script:
  - echo "Setting up environment..."
```

```
  # Script to set up environment

build_job:
  stage: build
  script:
    - echo "Building..."
    # Script to build the project

test_job:
  stage: test
  script:
    - echo "Running tests..."
    # Script to run tests
  environment:
    name: test
    url: https://example.com/test
  variables:
    # Job-specific variables
    TEST_VAR: "This is a test-specific variable"

deploy_job:
  stage: deploy
  script:
    - echo "Deploying..."
    # Script to deploy the project
  environment:
    name: production
    url: https://example.com/prod
  when: manual
  only:
    - master
  variables:
    # Job-specific variables
    DEPLOY_VAR: "This is a deploy-specific variable"
```

Pre-Defined variables

CI_COMMIT_REF_NAME: The branch or tag name for which the pipeline runs.
CI_COMMIT_SHORT_SHA: The shortened commit SHA (7 characters) for the latest commit.
CI_COMMIT_SHA: The full commit SHA for the latest commit.
CI_COMMIT_MESSAGE: The commit message for the latest commit.
CI_COMMIT_TITLE: The title of the commit for the latest commit.
CI_PROJECT_NAME: The name of the project in GitLab.
CI_PROJECT_NAMESPACE: The namespace (group or user) of the project in GitLab.
CI_PROJECT_PATH: The path of the project in GitLab (e.g., group/project).
CI_PROJECT_PATH_SLUG: The URL-friendly path of the project in GitLab.
CI_PIPELINE_ID: The unique identifier of the pipeline.
CI_PIPELINE_IID: The internal identifier of the pipeline.
CI_JOB_ID: The unique identifier of the current job.
CI_JOB_NAME: The name of the current job.
CI_JOB_STAGE: The stage of the current job.
CI_RUNNER_DESCRIPTION: The description of the runner executing the job.
CI_RUNNER_TAGS: The tags associated with the runner executing the job.
CI_RUNNER_VERSION: The version of the runner executing the job.
CI_SERVER_URL: The URL of the GitLab server.
CI_SERVER_NAME: The name of the GitLab server.
CI_SERVER_VERSION: The version of the GitLab server.

Building Docker Image

30 March 2024 09:04

```
stages:
   - build
docker_build_job:
   stage : build
   tags:
      - CI
      - Devops
   before_script:
      - echo "Build Started"
      - sudo docker login registry.gitlab.com -u autopilotdevops -p $Access_Token
   script:
      - sudo docker build -t registry.gitlab.com/autopilotdevops/autopilot-devops .
      - sudo docker push registry.gitlab.com/autopilotdevops/autopilot-devops
```

Only and Except

30 March 2024 09:06

only:
The only keyword specifies the conditions under which a job should be executed.
If the conditions specified with only are met, the job will run.
Multiple conditions can be specified using an array.
If no only or except keyword is defined, the job will run by default.

except:
The except keyword specifies the conditions under which a job should not be executed.
If the conditions specified with except are met, the job will not run.
Multiple conditions can be specified using an array.
If no only or except keyword is defined, the job will run by default.

```
stages:
  - test

test_job:
  stage: test
  script:
    - echo "Running tests..."
  only:
    - master  # Run only on the master branch
    - tags    # Run only on tagged commits
    - schedules  # Run only on scheduled pipelines
    - triggers   # Run only on pipelines triggered by a webhook or API call
    - web        # Run only on web pipeline triggers
    - api        # Run only on API pipeline triggers
    - merge_requests  # Run only on merge requests
    - branches@gitlab-org/gitlab  # Run only on branches of a specific forked repository
  except:
    - triggers   # Do not run on pipelines triggered by a webhook or API call
    - schedules  # Do not run on scheduled pipelines
    - web        # Do not run on web pipeline triggers
    - api        # Do not run on API pipeline triggers
```

Rules

30 March 2024 09:09

- The `rules` keyword allows defining conditions under which jobs should be included or excluded from a pipeline.
- Each rule consists of conditions and an associated action.
- Multiple rules can be defined, and they are evaluated in order.
- If no rules match, the job will not be executed.
- Rules can use predefined variables, expressions, and logical operators to define conditions.
- Actions associated with rules include `if`, `when`, `allow_failure`, `changes`, and `exists`.
- Cannot be used with only/except

Example:

```
stages:
  - test

test_job:
  stage: test
  script:
    - echo "Running tests..."
  rules:
    - if: '$CI_COMMIT_BRANCH == "master"'
    - if: '$CI_COMMIT_TAG =~ /^v\d+\.\d+\.\d+$/'
    - when: always
```

In this example:
- The `test_job` will run if either the branch of the commit is `master` or the commit is tagged with a version tag matching the pattern `vX.X.X`.
- Additionally, the job will always run regardless of the outcome of previous jobs in the pipeline.

Timeout

30 March 2024 09:10

Timeout in GitLab CI/CD:

- The `timeout` keyword is used to define the maximum amount of time a job is allowed to run.
- It specifies the duration in which the job must complete its execution, after which it will be automatically terminated. It should be less than runner timeout.
- Timeout can be defined at the job level or globally for all jobs in the pipeline.
- Timeout values can be specified in seconds, minutes, hours, or days.
- If a job exceeds the specified timeout duration, it will be marked as failed, and subsequent actions (such as cleanup or notifications) may be triggered.

Example:

```
stages:
  - build

build_job:
  stage: build
  script:
    - echo "Building the project..."
  timeout: 1 hour 30 minutes
```

In this example:
- The `build_job` has a timeout of 1 hour 30 minutes specified.
- If the job exceeds one hour 30 minutes of execution time, it will be automatically terminated.

When

30 March 2024 09:12

When in GitLab CI/CD:

- The `when` keyword is used to define the conditions under which a job should be executed.
- It allows controlling the execution flow of jobs based on the status of previous jobs or pipeline triggers.
- Different values for `when` include `on_success`, `on_failure`, `always`, `manual`, and `delayed`.
- `on_success` executes the job only if the previous jobs in the pipeline were successful.
- `on_failure` executes the job only if the previous jobs in the pipeline failed.
- `always` always executes the job, regardless of the status of previous jobs.
- `manual` requires manual intervention to run the job.
- `delayed` schedules the job to run after a specified duration.

Example:

```
stages:
  - test
  - deploy

test_job:
  stage: test
  script:
    - echo "Running tests..."
  when: on_success

deploy_job:
  stage: deploy
  script:
    - echo "Deploying the application..."
  when: manual
```

In this example:
- The `test_job` will run only if the previous jobs in the pipeline were successful.
- The `deploy_job` requires manual intervention to run and will not be executed automatically.

Allow Failure

30 March 2024 09:13

Allow Failure in GitLab CI/CD:

- The `allow_failure` keyword is used to indicate that a job is allowed to fail without causing the entire pipeline to fail.
- It is useful for jobs that are non-critical or experimental, where failure does not necessarily indicate a problem with the codebase.
- If a job marked as `allow_failure` fails, the pipeline will continue to execute, and subsequent jobs will still run.
- Jobs marked as `allow_failure` are displayed differently in the GitLab CI/CD interface to indicate that they are non-blocking.
- Although the job is allowed to fail, it will still be executed, and its output will be available for review.

Example:

```
stages:
  - test

test_job:
  stage: test
  script:
    - echo "Running tests..."
  allow_failure: true
```

In this example:
- The `test_job` is marked as `allow_failure`, indicating that it is allowed to fail without impacting the overall pipeline result.
- If `test_job` fails, the pipeline will continue to execute, and subsequent jobs will still run.

Concurrent Property

30 March 2024 09:14

Concurrent Property in GitLab Runner:

- The `concurrent` property in GitLab Runner configuration specifies the maximum number of job instances that can be run concurrently on the same Runner.
- It controls the level of parallelism, allowing multiple jobs to be executed simultaneously on a single Runner.
- By default, the `concurrent` property is set to the number of CPU cores available on the Runner machine.
- Setting a higher value for `concurrent` increases the number of jobs that can be processed concurrently, potentially improving pipeline execution speed.
- However, it's important to consider the hardware resources and performance limitations of the Runner machine when adjusting the `concurrent` property.
- Increasing the value of `concurrent` without sufficient resources may lead to resource contention, performance degradation, or job failures.

Example GitLab Runner Configuration (config.toml):

```
[[runners]]
  name = "My Runner"
  url = "https://gitlab.com/"
  token = "my-runner-token"
  executor = "shell"
  [runners.custom_build_dir]
  [runners.cache]
    [runners.cache.s3]
    [runners.cache.gcs]
  concurrent = 4
```

In this example:
- The `concurrent` property is set to 4, allowing the Runner to execute up to 4 job instances concurrently.
- Adjusting the `concurrent` property according to the hardware resources and workload requirements can optimize job execution and overall pipeline performance.

Needs

30 March 2024 09:16

Needs in GitLab CI/CD:

- The `needs` keyword is used to define dependencies between jobs in a GitLab CI/CD pipeline.
- It allows specifying which jobs must successfully complete before a particular job can start.
- With `needs`, you can create directed acyclic graphs (DAGs) of job dependencies, enabling complex pipeline workflows.
- Jobs specified in the `needs` keyword can be from the same or different stages in the pipeline.
- If a job specified in `needs` fails, the dependent job will not start, and the pipeline will fail.
- The `needs` keyword replaces the `dependencies` keyword, providing a more flexible and expressive way to define job dependencies.

Example:

```
stages:
  - build
  - test

build_job:
  stage: build
  script:
    - echo "Building the project..."

test_job:
  stage: test
  script:
    - echo "Running tests..."
  needs:
    - build_job
```

In this example:
- The `test_job` depends on the successful completion of the `build_job`.
- If `build_job` fails, `test_job` will not start, and the pipeline will fail.

Artifact

30 March 2024 09:17

Artifact in GitLab CI/CD:

- The `artifacts` keyword is used to specify files and directories generated by a job that should be passed to subsequent jobs or made available for download.
- It allows capturing and sharing build artifacts, test reports, logs, and other output produced during job execution.
- Artifacts can be defined at the job level to include specific files or directories, or they can be defined globally for all jobs in the pipeline.
- Artifacts are stored on GitLab's server and can be accessed by other jobs or pipelines within the same project.
- Artifacts can be used for various purposes, such as sharing build outputs, publishing documentation, archiving test results, etc.
- Artifacts can be downloaded manually from the GitLab UI or via the GitLab API for further analysis or debugging.
- The `exclude` keyword allows excluding specific files or directories from the artifacts to be uploaded.
- The `expire_in` keyword sets the expiration time for artifacts, after which they are automatically deleted from GitLab's server.

Example:

```
stages:
  - build

build_job:
  stage: build
  script:
    - make build
  artifacts:
    paths:
      - binaries/
      - reports/
    exclude:
      - logs/
    expire_in: 1 week
```

In this example:
- The `build_job` generates artifacts located in the `binaries/` and `reports/` directories, excluding the `logs/` directory.
- These artifacts will be available for download and can be passed to subsequent jobs in the pipeline.
- The artifacts will expire and be automatically deleted after 1 week.

Template

30 March 2024 09:20

```
stages:
  - build
  - test

.docker_template: &template
  tags:
    - CI
    - Devops
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
  timeout: 1 hours 30 minutes

docker_build_job:
  <<: *template
  stage : build
  before_script:
    - echo "Build Started"
    - sudo docker login registry.gitlab.com -u autopilotdevops -p $Access_Token
  script:
    - sudo docker build -t registry.gitlab.com/autopilotdevops/autopilot-devops .
    - sudo docker push registry.gitlab.com/autopilotdevops/autopilot-devops
  artifacts:
    paths:
      - $CI_PROJECT_DIR/

docker_test_job:
  <<: *template
  stage: test
  before_script:
    - echo "Image Verification Started"
  script:
    - sudo docker images
    - sudo docker image inspect registry.gitlab.com/autopilotdevops/autopilot-devops
```

Rules

30 March 2024 15:27

The "rules" keyword is used to define conditions for when jobs should be executed. It's a more flexible and expressive alternative to the "only" and "except" keywords. It also allows you to specify conditions based on different factors like branch names, tags, changes to files, variables, and more.

```
job1:
  script:
    - echo "This job will execute if any of the conditions are met"
  rules:
    - changes
    - when: always
    - exists: true
    - when: on_success


job2:
  script:
    - echo "This job will execute if all conditions are met"
  rules:
    - if: '$CI_COMMIT_BRANCH == "master" && $CI_PIPELINE_SOURCE == "web"'
```

Retry

03 April 2024 19:10

The "retry" keyword is used to specify the number of times a job should be retried if it fails. This can be useful for jobs that occasionally fail due to transient issues such as network problems or resource constraints.

```
job_name:
  script:
    - echo "This is my job"
  retry: 3
```

YML

04 April 2024 08:51

YAML stands for “YAML Ain’t Markup Language.” It is a data serialization language that is known for its simplicity, human readability, and ease of understanding.

YAML uses three node types:

- **Mappings** (Dictionaries): Unordered sets of key/value pairs, with unique keys.
    - animal: pets

  pets:
  - Cat
  - Dog
  - Goldfish

- **Sequences** (Arrays/Lists): Ordered series of nodes, which may contain duplicates or even themselves.
    - Cat
    - Dog
    - Goldfish

- **Scalars** (Variables): Data presented as Unicode characters, including strings, numbers, and booleans.
    integer: 25
    string: "25"
    float: 25.0
    boolean: true

Indentation: Indentation is crucial in YAML to indicate nesting, similar to Python. Tabs are not allowed; only spaces can be used. The number of spaces doesn’t matter as long as it’s consistent within the same level.
Adding . In front of job will disable the

CI/CD

04 April 2024 09:46

Continuous Integration (CI):
- Continuous Integration is a development practice where developers integrate code into a shared repository frequently, preferably several times a day.
- The main goal of CI is to detect and fix integration errors quickly by automating the build and test process.
- In GitLab, CI is achieved using GitLab CI/CD, a built-in feature that allows for the automation of the build, test, and deployment processes.

Continuous Delivery (CD):
- Continuous Delivery is an extension of Continuous Integration where every change that passes automated tests is made ready for deployment to production manually, allowing for additional checks or approvals before releasing the changes.
- Delivery through its pipelines feature, enabling the automation of the build, test, and deployment processes while providing the flexibility to control when changes are deployed to production.

Continuous Deployment (CD):
- Continuous Deployment is an extension of Continuous Integration where every change that passes automated tests is automatically deployed to production.
- The goal of CD is to release software to production quickly and frequently, reducing the time between writing code and making it available to users.
- GitLab supports Continuous Deployment through its pipelines feature, allowing for the automated deployment of code to various environments.